

1 # Claude Claude.md

2 title

3 author

4 tags

```
$ claude
# Reading CLAUDE.md ...
# Loading 5 skills ...
# 2 MCP Servers connected
# 3 hooks active
claude> Add a /users endpoint

✓ src/routes/users.ts
✓ src/services/userService.ts
✓ tests/users.test.ts - 8/8 passing
✓ prettier - formatted
✓ eslint - no issues
```

A SOFTWARE ENGINEER'S FIELD GUIDE

CLAUDE in ONE DAY

Set up **CLAUDE.md**, **Skills**, **MCP Servers**, **Hooks**, and **Agentic Workflows**
— from zero to fully configured in 8 hours

CLAUDE.md

Skills

MCP

Hooks

Agents

Subagents

Agentic Loops

1 # Claude Claude.md
2 title
3 author
4 tags

1 # Claude Claude.md
2 title
3 author
4 tags

Claude in One Day: A Software Engineer's Field Guide
Copyright © 2026 Alejandro Pronotti. All rights reserved.

No part of this book may be reproduced without written permission from the author.

Claude, Claude Code, and Anthropic are trademarks of Anthropic, PBC.

This book is not affiliated with or endorsed by Anthropic.

First Edition — March 2026

Published independently

CONTENTS

Preface — Why This Book, Why Now

Chapter 1 The Mental Model Shift

Chapter 2 Installation and First Session

Chapter 3 CLAUDE.md — Your Engineering Contract

Chapter 4 Skills — Teaching Claude Your Patterns

Chapter 5 Commands, Agents, and Agentic Workflows

Chapter 6 MCP Servers — External Superpowers

Chapter 7 Hooks — Automated Quality Gates

Chapter 8 Real Project Walkthrough

Chapter 9 Team Configuration and Collaboration

Chapter 10 The One-Day Setup Checklist

Appendix A — Complete CLAUDE.md Template

Appendix B — Skill Library Starter Kit

Appendix C — Resources and Links

CHAPTER —

PREFACE

This book exists because I watched dozens of talented engineers — people who build distributed systems, design APIs, and debug race conditions in their sleep — struggle to get real value from Claude. Not because the tool is bad. Because the default experience hides 90% of the power behind configuration files that nobody tells you about.

I am not going to teach you prompt engineering. You do not need 100 copy-paste prompts. What you need is to understand the configuration layer that sits between Claude and your codebase — and to set it up properly, once, in a single focused day.

By tonight, Claude will know your stack, your conventions, your test commands, and your code review checklist. It will run your linter after every edit. It will follow your team's architecture patterns without being told twice. This is not magic. It is configuration.

Who this book is for:

- Software engineers who have used ChatGPT or Copilot but feel they are only scratching the surface
- Developers who installed Claude Code, typed a few prompts, and thought "this is just a chatbot in my terminal"
- Tech leads who want to standardize AI-assisted development across a team
- Anyone who has 8 hours and wants to fundamentally change how they write software

What you will have by end of day:

A global CLAUDE.md encoding your personal engineering style — the base layer that persists across every project. A project-level CLAUDE.md with your stack, architecture, and conventions that layers on top without erasing the global one. A `.claude/rules/` directory with modular team standards that travel between codebases. At least 5 custom skills covering your most common workflows. Hooks that auto-format, lint, and test after every Claude action. MCP servers connecting Claude to your databases, APIs, and tools. And a mental model for how these layers combine — and why that matters for keeping team directives consistent across projects.

"The difference between a generic AI assistant and a custom-configured coding partner is not the model — it is the files around it."

Companion Repository

Every template, configuration file, and skill from this book is available as a ready-to-clone repository. Do not retype — fork, customize, and start building:

github.com/apronotti/claude-in-one-day

The repo includes the global CLAUDE.md template, a complete project-level `.claude/` directory with all five skills from Appendix B, example slash commands, agent definitions, hook configurations, the team dotfiles setup script from Chapter 9, and the `.claude/rules/` templates. Star the repo to get notified when templates are updated for new Claude features.

CHAPTER 1

THE MENTAL MODEL SHIFT

Before we touch a single config file, we need to rewire how you think about AI-assisted coding. If you come from traditional software engineering, your instinct is to think of Claude as a tool — like a compiler, a linter, or a smarter autocomplete. That framing will limit everything you do with it.

From Tool to Collaborator

Think of Claude not as a tool you operate, but as a junior-to-mid engineer who just joined your team. They are fast, tireless, and have read every Stack Overflow answer ever posted. But they do not know your codebase, your conventions, your deployment process, or that one weird thing about how your ORM handles polymorphic relations. Without onboarding, even brilliant engineers write code that does not fit.

The configuration files we will build today are that onboarding. CLAUDE.md is the team wiki — layered between global standards and project context. The `.claude/rules/` directory holds modular directives that persist across projects. Skills are the runbooks. Hooks are the CI checks. MCP servers are the tool access. By the end of today, Claude will be a properly onboarded member of your engineering team.

The Configuration Hierarchy: Layering, Not Overwriting

This is the single most misunderstood concept in Claude Code configuration, so let us get it right from the start. The configuration hierarchy does not work like variable overwriting, where a project file replaces the global one. It works like CSS specificity — layers accumulate. Every layer is active simultaneously in the same session.

```

~/.claude/
  CLAUDE.md          <- Layer 1: personal defaults
  settings.json      <- Global permissions, model prefs
  skills/            <- Skills available in ALL projects

your-project/
  CLAUDE.md          <- Layer 2: project context (ADDS to L1)
  .claude/
    settings.json    <- Project hooks, permissions
    settings.local.json <- Layer 3: personal escape hatch
    rules/           <- Layer 2b: modular auto-loaded rules
    commands/        <- Slash commands (/deploy, /review)
    skills/           <- Project-specific skills
    agents/           <- Custom agent personas

```

If your global CLAUDE.md says "never use `*` imports in Python" and your project CLAUDE.md says "use SQLAlchemy for database access," Claude sees **both** instructions at the same time. The project file does not erase the global one — it enriches it. Only when instructions explicitly conflict (global says tabs, project says spaces) does the more specific layer win.

This has a critical implication for teams: your organization-wide engineering standards belong in the global layer where they persist across every project. Project-level files should only contain what is unique to that specific codebase. We will build on this pattern extensively in Chapters 3 and 9.

What Changes Today

By the end of this chapter, you should have internalized three principles that will guide every configuration decision:

- **Context is everything.** Claude's output quality is directly proportional to the context you provide. Not in the prompt — in the configuration files it reads before you type a single word.
- **Layers accumulate, not replace.** Global standards, project context, and rules files all live simultaneously in Claude's context. Put each directive in the right layer once, and it persists everywhere it should.

-
- **Explicit beats implicit.** If you want Claude to use type hints everywhere, say so in CLAUDE.md. If you want it to run tests after editing test files, set up a hook. Never assume Claude will infer your preferences.
 - **Progressive disclosure.** Start with a minimal config. Add layers as you discover friction. The worst Claude setups are the ones with 500-line CLAUDE.md files written on day one. Grow it organically.

CHAPTER 2

INSTALLATION AND FIRST SESSION

This chapter gets Claude Code running on your machine in under 15 minutes. No fluff — just the commands and the decisions you need to make.

Prerequisites

- Node.js 18+ (required for Claude Code — check with `node --version`)
- A Claude Pro, Max, or Team subscription — or an Anthropic API key
- A terminal you are comfortable in (iTerm2, WezTerm, Ghostty, Windows Terminal)
- Git installed and configured

Install Claude Code

```
npm install -g @anthropic-ai/claude-code

# Verify installation
claude --version

# Launch for the first time
cd your-project
claude
```

On first launch, Claude will ask you to authenticate. If you have a Pro/Max subscription, it will open your browser. If you are using an API key, you can pass it via environment variable `ANTHROPIC_API_KEY`.

Your First Interaction — But Do Not Get Comfortable

Go ahead and ask Claude something about your project. It will work. It will read files, suggest edits, maybe even fix a bug. And you might think: "This is fine. Why do I need configuration files?"

Here is why. Ask Claude to write a new API endpoint in your project. Watch what happens. It will probably produce correct code. But it will not match your naming conventions. It will not use your preferred error handling pattern. It will not know that your team uses a specific ORM wrapper. It will not run your linter afterward. It is writing code for a generic project, not yours.

That gap between "correct code" and "code that belongs in your project" is exactly what the next eight chapters eliminate.

Choose Your Access Mode

Claude Code offers three surfaces beyond the terminal, and your CLAUDE.md, skills, and settings work across all of them:

- **Terminal** — The core experience. Best for focused, context-heavy work.
- **VS Code / JetBrains extensions** — Claude in your editor sidebar. Good for file-centric work.
- **Claude Desktop / Web** — Skills also work here, so you can define them once and use everywhere.

CHAPTER 3

CLAUDE.md — YOUR ENGINEERING CONTRACT

This is the most important file in your entire Claude setup. CLAUDE.md is a markdown file that Claude reads automatically at the start of every session. You do not paste it. You do not reference it. It is loaded into context every single time, silently shaping every response.

How Layering Actually Works

Before writing anything, internalize this: layers accumulate, they do not replace. When Claude starts a session, it reads your global `~/.claude/CLAUDE.md` first, then your project `CLAUDE.md`, then any files in `.claude/rules/`. All of them are active simultaneously. Think of it like building a sandwich, not swapping plates.

What Claude sees in every session:

```
+-- ~/.claude/CLAUDE.md (always loaded) -----+
| "Never use * imports. Add type hints to all   |
| public functions. Be direct. Use gh for Git." |
+-----+
+
+-- project/CLAUDE.md (adds on top) -----+
| "Stack: Python 3.12, FastAPI, SQLAlchemy.     |
| Services never import from routes."          |
+-----+
+
+-- .claude/rules/*.md (auto-loaded context) ---+
| security-rules.md, api-conventions.md, etc.  |
+-----+
=
Claude knows ALL of the above simultaneously
```

This is why project CLAUDE.md files do not destroy your team directives. Your global standards persist across every project. The project file only adds what is specific to that codebase. Only when two

instructions directly contradict each other does the more specific layer take priority.

The Global CLAUDE.md — Team DNA

Create `~/.claude/CLAUDE.md` for standards that apply everywhere, regardless of project. Keep this short — 20-40 lines maximum. It consumes context tokens on every session.

```
# Global Preferences

## Communication Style
- Be direct. Skip preamble and filler.
- When I ask for code, give me code. Explain only if I ask.
- If something is wrong with my approach, say so. Do not be sycophantic.
- Use gh CLI for all GitHub operations.

## Code Style (all projects)
- Add type hints to all public functions
- Never use `from module import *`
- Always handle exceptions explicitly, never pass
- Write tests alongside implementation, not after
```

Key insight: for teams, store this file in a shared dotfiles repository. Every engineer clones the same global CLAUDE.md. Personal tweaks go in the project-level settings.local.json, not here. This is how you maintain team directives across all projects.

The Project CLAUDE.md — Codebase Context

Place `CLAUDE.md` in your project root. This file should contain **only** what is unique to this project — its stack, architecture, and conventions. Do not repeat global standards here. Since layers accumulate, your global rules are already active.

```
# Project: payments-api

## Quick Facts
- Stack: Python 3.12, FastAPI, Pydantic v2
- Database: PostgreSQL 16 via SQLAlchemy 2.0
- Test: `pytest` (with pytest-asyncio)
- Lint: `ruff check .` and `ruff format .`
- Run: `uvicorn app.main:app --reload`

## Architecture
- app/routes/      -> FastAPI route handlers
- app/services/    -> Business logic (no HTTP deps)
- app/repos/       -> Database access via SQLAlchemy
- app/middleware/  -> Auth, error handling, logging
- tests/          -> Mirrors app/ structure

## Conventions (project-specific)
- Services never import from routes
- All responses use ApiResponse[T] wrapper
- Errors are typed: AppError(Exception) with code
- Dates are ISO 8601 strings in API, datetime in code
- Use Pydantic models for request validation

## Review Checklist (before committing)
- [ ] Tests pass: pytest
- [ ] Lint passes: ruff check .
- [ ] No print() in production code
- [ ] Error cases handled and typed
- [ ] Migration created if schema changed
```

The `.claude/rules/` Directory — Modular Standards

This is the secret weapon for maintaining team directives across projects. Instead of cramming everything into one monolithic `CLAUDE.md`, you can drop individual markdown files into `.claude/rules/`. Each file auto-loads when its context is relevant. This gives you composable, maintainable standards.

```
.claude/rules/
security-rules.md      <- Always loaded
api-conventions.md    <- Always loaded
testing-standards.md  <- Always loaded
react-patterns.md     <- Loaded for frontend work
```

The power of rules files is portability. Your `security-rules.md` can be identical across every project in your organization — symlinked from a central repo, copied by a setup script, or managed as a git submodule. The project `CLAUDE.md` stays lean and project-specific while team standards persist through the rules directory.

```
# .claude/rules/security-rules.md

## Security (applies to all projects)
- Never log secrets, tokens, or PII
- Always parameterize SQL queries
- Validate and sanitize all user input
- Use crypto.randomUUID() not Math.random()
- HTTPS for all external requests
- Never commit .env files or hardcoded secrets
```

What NOT to Put in CLAUDE.md

A common mistake is treating `CLAUDE.md` like a 500-line specification document. Remember: every line consumes context tokens. Bloated files push out space for actual code and reasoning. Follow these guidelines:

- **No tutorials.** Claude already knows how SQLAlchemy works. Tell it YOUR SQLAlchemy conventions.
- **No obvious rules.** "Write clean code" is noise. "Services return Result[T, AppError], never raise" is signal.
- **No file contents.** Do not paste your `pyproject.toml`. Claude can read it directly.
- **No aspirational rules.** Only include conventions your team actually follows today.

-
- **No duplicated globals.** If it is in your global CLAUDE.md or a rules file, do not repeat it in the project file. Layers accumulate — repetition wastes tokens.

The golden rule: if you would tell a new hire on day one, it belongs in CLAUDE.md. If it is in your docs wiki, it does not. If it applies to every project, it goes in global or in .claude/rules/, not in the project file.

CHAPTER 4

SKILLS — TEACHING CLAUDE YOUR PATTERNS

If CLAUDE.md is the onboarding document, skills are the runbooks. A skill is a folder containing a SKILL.md file (plus optional supporting files) that gives Claude step-by-step instructions for a specific workflow. Claude loads skills automatically when it detects a matching task, or you can invoke them explicitly with a slash command.

Anatomy of a Skill

Every skill lives in a directory. The directory name becomes the skill identifier. Inside it, **SKILL.md** contains YAML frontmatter (how the skill runs) and markdown content (what Claude does):

```
.claude/skills/  
  api-endpoint/  
    SKILL.md          <- Skill definition  
    template.py       <- Supporting file  
  code-review/  
    SKILL.md  
    checklist.md      <- Reference material
```

Your First Skill: API Endpoint Generator

This skill encodes your team's exact pattern for creating new API endpoints. Save it as **.claude/skills/api-endpoint/SKILL.md**:


```

---
name: api-endpoint
description: >
    Create a new API endpoint following project
    conventions. Use when asked to add a route,
    endpoint, or API method.
---

# New API Endpoint

When creating a new endpoint, always:

1. Create route handler in app/routes/
2. Create service method in app/services/
3. Create repo method if DB access needed
4. Add Pydantic model for request validation
5. Add pytest test covering happy path + errors
6. Use ApiResponse[T] wrapper for all responses
7. Register route in app/routes/__init__.py

## Error Handling Pattern
Services return Result[T, AppError].
Routes convert to HTTP status codes.
Never raise from service layer.

```

Two Types of Skills

Reference skills add context that Claude applies to your current work. Think conventions, patterns, and style guides. These run inline — Claude uses them alongside your conversation.

Task skills give Claude step-by-step instructions for a specific action. Deployments, commit workflows, code generation pipelines. These often run as subagents — separate context windows that keep your main conversation clean.

```
---
name: deep-research
description: Research a topic thoroughly
context: fork
agent: Explore
---

Research $ARGUMENTS thoroughly:
1. Find relevant files using Glob and Grep
2. Read and analyze the code
3. Summarize findings with file references
```

Five Skills Every Engineer Should Build on Day One

- **code-review** — Your team's review checklist as a skill. Claude checks its own work.
- **commit** — Your commit message format, branch naming, and pre-commit checks.
- **test-writer** — Your testing conventions: what to test, how to mock, naming patterns.
- **debug** — Your debugging workflow: where logs go, how to reproduce, how to isolate.
- **document** — Your documentation standards: JSDoc format, README structure, ADR template.

CHAPTER 5

COMMANDS, AGENTS, AND AGENTIC WORKFLOWS

This chapter covers the three mechanisms that turn Claude from a reactive assistant into an autonomous collaborator: slash commands for repeatable workflows, custom agents for specialized personas, and agentic loops that let Claude plan, execute, verify, and iterate without waiting for your input at every step.

Slash Commands — Your Inner Loop

Slash commands are single-file prompts stored in `.claude/commands/` that you invoke by typing `/command-name` in Claude Code. If you do something more than twice a day, it should be a slash command.

Slash Commands vs Skills: When to Use Which

Slash commands are for workflows YOU trigger explicitly — they have great terminal discovery and autocomplete. Skills are for workflows CLAUDE triggers automatically based on task detection. The difference is packaging and UX: commands are single files with great terminal integration; skills are directories that can bundle supporting files and templates.

```
# .claude/commands/review-pr.md
```

Review the current PR against our standards:

1. Run `gh pr diff` to get the changes
2. Check each file against our code style
3. Verify test coverage for changed code
4. Check for security issues (SQL injection, XSS, auth bypass)
5. Verify error handling completeness
6. Output a structured review with:
 - MUST FIX: blocking issues
 - SHOULD FIX: improvements
 - NICE TO HAVE: suggestions
 - APPROVED: if no blocking issues

Custom Agents — Specialized Personas

Agents are pre-configured Claude instances with specific models, instructions, and tool access. They live in `.claude/agents/` and excel at tasks that need a different mindset from your main coding session.

```
# .claude/agents/reviewer.md
---
model: opus
description: Senior code reviewer
---
```

You are a critical senior engineer reviewing code. Your job is to find problems, not praise.

Focus on:

- Logic errors and edge cases
- Performance implications
- Security vulnerabilities
- Maintainability concerns

For each issue, provide: file, line, severity, explanation, and a suggested fix.

Practical Agent Setup

A production-ready setup usually has 3-4 agents. Following the pattern used by experienced teams:

- **Reviewer agent** (Opus) — Finds problems in code with senior-engineer rigor
 - **Debugger agent** (Sonnet) — Fast, focused bug tracking and isolation
 - **Feature builder** (Opus) — Implements new features following your patterns
 - **Planner agent** (Opus) — Breaks large tasks into context-window-sized chunks
-

The Agentic Mental Model

Everything above — commands, agents, skills — becomes dramatically more powerful once you understand the concept that ties them together: **agentic workflows**. This is the single biggest mental shift for traditional software engineers, and the one most books skip over.

In traditional coding, you operate in a request-response loop. You type a prompt, Claude responds, you evaluate, you type another prompt. You are the orchestrator. Claude is a function you call. This works fine for quick questions and small edits. But it breaks down completely for real engineering tasks — building a feature, refactoring a module, debugging a complex issue.

In an agentic workflow, you define the **goal** and the **constraints**. Claude handles the planning, execution, verification, and iteration. You shift from line-by-line pilot to air traffic controller. Your job becomes: define the destination, set the boundaries, and review the result.

TRADITIONAL (prompt-response):

```
You: "Add a /users endpoint"
Claude: [writes route file]
You: "Now add the service"
Claude: [writes service file]
You: "Now add tests"
Claude: [writes tests]
You: "Run the tests"
Claude: [runs, 2 fail]
You: "Fix them"
...10 more exchanges...
```

AGENTIC (goal + constraints):

```
You: "Add a /users CRUD endpoint following
      our conventions. Include tests. Make
      sure everything passes before stopping."
Claude: [plans] -> [writes route] -> [writes
      service] -> [writes repo] -> [writes
      tests] -> [runs tests] -> [fixes 2
      failures] -> [runs again] -> [all
      pass] -> [runs linter] -> [done]
You: [review the complete result]
```

The second approach is not just faster — it produces better results. When Claude handles the full loop, it maintains context across all the files, catches its own inconsistencies, and does not lose track of the original goal between exchanges.

Anatomy of an Agentic Loop

Every agentic workflow follows the same fundamental cycle, regardless of the task. Understanding this cycle lets you configure Claude to execute it well:

THE AGENTIC LOOP:

1. PLAN

Claude reads your request + CLAUDE.md + rules + relevant skills. It decomposes the task into steps.

2. EXECUTE

Claude works through steps using tools: read files, write code, run commands. Hooks fire after each tool use (format, lint, test).

3. VERIFY

Claude checks its own work: runs tests, reviews output, compares against your conventions from CLAUDE.md.

4. ITERATE or STOP

If verification fails, loop back to step 2.
If it passes, present the result.

Your config files shape every stage:

CLAUDE.md	-> shapes the PLAN
Skills	-> guide the EXECUTE
Hooks	-> enforce during EXECUTE
Agents	-> run VERIFY as a second opinion

This is where all the configuration work from the previous chapters pays off. Your CLAUDE.md ensures the plan aligns with your architecture. Your skills guide execution patterns. Your hooks enforce formatting and testing automatically. Your reviewer agent provides verification. Each config file is a control surface on the agentic loop.

Subagents — Parallel Brains

For complex tasks, Claude can spawn subagents — separate context windows that work on different parts of a problem simultaneously. The lead agent coordinates the work, assigns subtasks, and merges results. This is the most powerful agentic pattern, and it maps directly to how experienced engineering teams work.

ORCHESTRATOR PATTERN:

```
Lead Agent (your main session)
|
+--> Subagent 1: Research
|   (Explore agent, read-only tools)
|   -> returns: findings summary
|
+--> Subagent 2: Implementation
|   (general-purpose, full tool access)
|   -> returns: code changes
|
+--> Subagent 3: Review
|   (Opus model, critical persona)
|   -> returns: issues found
|
Lead: merges, resolves conflicts, presents
```

Why separate context windows matter: each subagent gets a clean context without the accumulated noise of your main conversation. A research agent can deeply explore the codebase without polluting the implementation context. A reviewer agent can evaluate code without being biased by having written it. One agent can *cause* bugs and another (same model, fresh context) can *find* them — this is test-time compute at work.

Enabling Agentic Workflows in Practice

You do not need special configuration to go agentic. Claude Code is agentic by default — it already plans, executes, and verifies. What your configuration does is make those loops **reliable**. Here is how each config file contributes:

- **CLAUDE.md** — Tells Claude what "done" looks like. Without clear conventions, the loop terminates too early or goes in circles.
- **Skills** — Provide step-by-step guides so execution follows your patterns, not generic ones.
- **Hooks** — Act as automated gates within the loop. Auto-format after edits, run tests after changes, block destructive operations. These

are the guardrails that make autonomy safe.

- **Agents** — Let you run verification as a separate agent with a critical persona, catching issues the implementation agent misses.
- **MCP Servers** — Give the agentic loop access to external tools: databases, APIs, search. Without these, Claude can only reason about files.

When NOT to Go Agentic

Agentic workflows are not always the right choice. There is a cost to autonomy: more tokens consumed, longer execution time, and the risk of Claude going down an unproductive path. Knowing when to stay in direct control is as important as knowing when to let go.

- **Quick questions and lookups** — "What does this function do?" does not need a plan-execute-verify cycle. Just ask.
- **Small, well-defined edits** — "Rename this variable" or "add a null check here" is faster with direct prompting.
- **Exploration and learning** — When you are trying to understand unfamiliar code, you want interactive back-and-forth, not autonomous execution.
- **Tasks smaller than the overhead** — If the task takes Claude 30 seconds with one prompt, do not spin up agents and subagents. Vanilla Claude Code is often better for small tasks.

A useful heuristic: if the task touches more than 3 files or requires more than 5 back-and-forth exchanges to complete, go agentic. If it is smaller, stay direct. When in doubt, start direct and say "use subagents" if you feel the task growing.

Context Management — The Hidden Skill

The biggest practical challenge with agentic workflows is context window management. Claude has a finite context window, and long agentic sessions fill it up. When context gets too full, Claude's output quality degrades — this is sometimes called "context rot." Manage it

actively:

- **/compact** — Manually compress context at around 50% capacity. Do not wait until it is full.
- **/clear** — Reset context entirely when switching to a new task mid-session.
- **Subagents for isolation** — Offload research and review to subagents so your main context stays clean and focused on implementation.
- **Phase-gated plans** — For large features, break work into phases. Complete and commit each phase before starting the next, resetting context between phases.

The engineers who get the most from agentic Claude are not the ones who write the longest prompts. They are the ones who keep their context clean, their phases small, and their configuration files doing the heavy lifting so Claude does not need to be told twice.

CHAPTER 6

MCP SERVERS — EXTERNAL SUPERPOWERS

Model Context Protocol (MCP) servers are the bridges connecting Claude to the outside world. Think of them as USB ports for your AI — they let Claude interact with databases, APIs, browsers, and external services without leaving your coding session.

How MCP Works

An MCP server exposes tools and resources that Claude can call. When Claude needs to query your database, search your documentation, or check your CI status, it makes a request to the appropriate MCP server. You configure which servers are available in your `settings.json`.

Essential MCP Servers for Engineers

- **GitHub MCP** — PR management, issue creation, repository search. But honestly, the `gh` CLI in `CLAUDE.md` often works better for simple operations.
- **PostgreSQL / SQLite MCP** — Direct database queries. Claude can inspect schema, run queries, and debug data issues without you copying output back and forth.
- **Brave Search MCP** — Web search from within Claude Code. Useful for looking up documentation, checking latest package versions, or researching error messages. Free and takes about five minutes to set up.
- **Filesystem MCP** — Controlled access to specific directories outside your project.

Configuration

```
// .claude/settings.json
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-postgres",
        "postgresql://localhost:5432/mydb"
      ]
    },
    "brave-search": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-brave-search"
      ],
      "env": {
        "BRAVE_API_KEY": "your-key-here"
      }
    }
  }
}
```

Start with one MCP server. The database one is usually the highest-value addition. Add more as you discover specific needs, not because a blog post said you should.

MCP in the Agentic Loop

MCP servers become especially powerful in agentic workflows (Chapter 5). When Claude is autonomously building a feature, it can query your database to understand the current schema, search the web for documentation on an unfamiliar library, and check your CI status — all without breaking the agentic loop to ask you for information. Without MCP, Claude can only work with files in your project directory. With MCP, it can reason about your entire development ecosystem.

A practical tip: put MCP server configuration in `.claude/settings.json` (committed to git) for servers the whole team uses, like your shared database. Put personal servers — like your own

Brave Search API key — in `.claude/settings.local.json` (gitignored). This follows the same layering principle from Chapter 3.

CHAPTER 7

HOOKS — AUTOMATED QUALITY GATES

Hooks are shell commands that run automatically before or after Claude Code actions. They are your automated safety net — the guardrails that catch problems before they reach your codebase.

Hook Types

- **PreToolUse** — Runs before Claude uses a tool (edit, write, bash). Can block the action.
- **PostToolUse** — Runs after a tool completes. Perfect for formatting and linting.
- **Stop** — Runs when Claude finishes a turn. Use to verify or nudge continuation.

The Three Hooks Every Project Needs

1. Auto-format after every edit

```
// .claude/settings.json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "ruff format $FILE 2>/dev/null || true",
            "timeout": 10
          }
        ]
      }
    ]
  }
}
```

2. Block edits on main branch

```
{
  "matcher": "Edit|Write",
  "hooks": [
    {
      "type": "command",
      "command": "[ \"$(git branch --show-current)\" != \"main\" ] || echo bloc
      "timeout": 5
    }
  ]
}
```

3. Run tests when test files change

This hook watches for edits to test files and automatically runs the relevant tests, catching regressions immediately. Claude generates well-formatted code 90% of the time — hooks handle the last 10% to avoid CI failures.

Hooks are deterministic enforcement. Unlike instructions in CLAUDE.md (which Claude might occasionally miss), hooks always run. Use CLAUDE.md for guidance; use hooks for guarantees.

Hooks in the Agentic Loop

Hooks play a critical role in agentic workflows (Chapter 5). When Claude is running autonomously — planning, writing code, running tests, iterating — hooks fire after every single tool use. This means your auto-formatter runs after every edit, your linter catches issues in real time, and your branch protection prevents mistakes even when you are not watching. Hooks are what make it safe to let Claude operate with more autonomy. Without them, agentic workflows require constant supervision. With them, you can trust the loop to enforce your standards mechanically.

CHAPTER 8

REAL PROJECT WALKTHROUGH

Let us put everything together. We will set up Claude for a real project — a REST API for a task management application. This walkthrough follows the exact sequence you would follow in practice.

Hour 1-2: Foundation (Layered CLAUDE.md)

Start with the global layer. If your team already has a shared dotfiles repo (Chapter 9), run the install script to sync `~/ .claude/CLAUDE.md`. If not, create your own with 20-40 lines of personal standards — communication style, universal code conventions, and tool preferences. This layer will be active in every project from now on.

Next, open your project root. Create the project-level CLAUDE.md following the structure from Chapter 3. Remember: this file **adds to** your global file, it does not replace it. So only include what is specific to this codebase — stack, architecture, directory structure, build commands, and project-specific conventions. Do not repeat anything from your global file.

If you have team-wide rules (security, testing standards, API conventions), create a `.claude/rules/` directory and add them as separate markdown files. These auto-load alongside both CLAUDE.md files.

Run Claude and ask it to add a feature. Watch what it gets wrong. Whatever it gets wrong, decide which layer the fix belongs in: global (applies everywhere), project CLAUDE.md (this codebase only), or rules (team standard). This feedback loop is the fastest way to build a useful config.

Hour 3-4: Skills and Commands

Think about the three tasks you do most frequently. For most engineers, these are: creating new endpoints/components, writing tests, and reviewing code. Build a skill for each one. Start with the skeleton from Chapter 4 and customize it to your patterns.

Then add slash commands for your daily workflows: `/commit` for your commit convention, `/review-pr` for pull request review, and `/deploy-staging` if you have a staging environment.

Hour 5-6: Hooks and MCP

Set up auto-formatting as a PostToolUse hook — this is the single highest-value hook you can add. Then configure one MCP server (usually your database). Test both by asking Claude to write and edit some code, confirming the formatter runs and database access works.

Hour 7-8: Agents and Refinement

Create your reviewer agent. Ask Claude to build a feature, then invoke the reviewer agent on the result. The quality difference is immediately visible. Spend the remaining time iterating: tweak your CLAUDE.md based on what you have observed, add edge cases to your skills, and adjust hook timeouts.

The goal is not perfection today. The goal is a solid foundation that compounds over time. Every hour you invest in configuration pays back 10x over the next month.

CHAPTER 9

TEAM CONFIGURATION AND COLLABORATION

Individual configuration is powerful. Team configuration is transformative. When every engineer on your team has the same standards enforced through Claude's configuration layer, you get consistency that no code review process can match.

But here is the question every tech lead asks: "If each project has its own CLAUDE.md, how do I maintain team-wide directives across all projects?" This chapter gives you three battle-tested patterns.

The Core Problem: Standards That Survive

You have security rules, error handling philosophies, commit conventions, and code review criteria that apply to every project your team touches. You cannot copy-paste these into every project's CLAUDE.md — that creates drift, duplication, and maintenance nightmares. You need standards that persist automatically, without engineers remembering to include them.

The solution comes from understanding layering (Chapter 3): since global and project configs accumulate rather than replace each other, you have multiple injection points for team standards.

Pattern 1: The Shared Dotfiles Repository

This is the simplest and most reliable pattern. Your team maintains a single git repository containing the global `~/.claude/` directory. Every engineer clones it and symlinks or copies it to their home directory.

```
# Team dotfiles repo structure
team-claude-config/
  CLAUDE.md          <- Team-wide standards
  settings.json      <- Shared permissions
  skills/
    code-review/SKILL.md <- Team review checklist
    commit/SKILL.md    <- Commit conventions
    security-audit/SKILL.md
  install.sh         <- Symlinks to ~/.claude/
```

```
# install.sh
#!/bin/bash
REPO_DIR=$(cd "$(dirname "$0")" && pwd)

# Back up existing config
[ -d ~/.claude ] && mv ~/.claude ~/.claude.bak

# Symlink the team config
ln -sf "$REPO_DIR" ~/.claude

echo "Team Claude config linked."
echo "Personal overrides: edit project-level"
echo "settings.local.json (gitignored)."
```

Now every engineer on the team has identical global standards. When the team updates **CLAUDE.md** in the repo, everyone gets the change on their next **git pull**. Personal preferences (like model choice or verbosity) go in each project's **.claude/settings.local.json**, which is gitignored.

Pattern 2: The **.claude/rules/** Directory with Shared Sources

For teams that need more granularity, the **.claude/rules/** directory is the power tool. Each markdown file in this directory auto-loads into every Claude session for that project. The key insight: these files can come from a shared source.

```
# Option A: Git submodule
cd your-project
git submodule add git@github.com:your-org/\
  claude-standards.git .claude/rules/team

# Result:
# .claude/rules/team/security.md    <- from org repo
# .claude/rules/team/api-style.md   <- from org repo
# .claude/rules/local-overrides.md  <- project-only
```

```
# Option B: Setup script in your project
# scripts/setup-claude.sh
#!/bin/bash

STANDARDS_REPO="git@github.com:your-org/\
  claude-standards.git"
RULES_DIR=".claude/rules/team"

if [ ! -d "$RULES_DIR" ]; then
  git clone "$STANDARDS_REPO" "$RULES_DIR"
else
  cd "$RULES_DIR" && git pull && cd -
fi

echo "Team rules synced."
```

This pattern lets each project keep its own CLAUDE.md lean (only project-specific context) while inheriting all team standards through the rules directory. Update the central repo, and every project gets the new rules on next sync.

Pattern 3: Organization-Provisioned Skills (Team/Enterprise)

If your organization is on a Claude Team or Enterprise plan, administrators can provision skills centrally for all users. These appear with a team badge in each engineer's skills list. Individuals can toggle them on or off, but cannot modify the content. This is the zero-friction option — no git submodules, no setup scripts, no symlinks. Standards are pushed from admin to every team member automatically.

What to Commit vs What to Keep Personal

This decision is critical and most teams get it wrong. Here is the clean split:

- **Commit to git:** Project CLAUDE.md, .claude/commands/, .claude/skills/, .claude/agents/, .claude/settings.json (hooks, shared MCP servers), .claude/rules/ (team standards)
- **Gitignore:** .claude/settings.local.json (personal overrides, API keys, model preferences), .claude/auto-memory/ (Claude's learning from your sessions)

The Layering Cheat Sheet

Use this quick reference to decide where each type of directive belongs:

WHERE DOES THIS RULE GO?

"Never use * imports in Python"
-> Global CLAUDE.md (applies everywhere)

"Use SQLAlchemy for database access"
-> Project CLAUDE.md (project-specific)

"Never log PII or secrets"
-> .claude/rules/security.md (team standard)

"I prefer Opus over Sonnet"
-> .claude/settings.local.json (personal)

"All commits use conventional format"
-> Global skill: ~/.claude/skills/commit/

"This project deploys to AWS ECS"
-> Project skill: .claude/skills/deploy/

Team Onboarding: Day One Productivity

When a new engineer joins and clones your repo, they get your project CLAUDE.md, skills, commands, agents, hooks, and rules directory

automatically. They run `install.sh` from the team dotfiles repo to get the global layer. They create a `.claude/settings.local.json` for personal preferences. Total setup time: under 10 minutes. Claude already knows your codebase conventions before the new hire does.

Evolving Configuration as a Team

Treat your CLAUDE.md, rules, and skills like code. Review changes in PRs. If someone adds a rule, it should have the same scrutiny as a linter rule change — because functionally, it is one. Run a monthly retro on your Claude configuration: what rules are working, what is Claude still getting wrong, what new skills would save the most time.

The teams getting the most from Claude are not the ones with the longest CLAUDE.md files. They are the ones with the cleanest separation between global standards, project context, team rules, and personal preferences — each in its proper layer, none duplicated.

CHAPTER 10

THE ONE-DAY SETUP CHECKLIST

You have read the book. Now execute. This checklist maps to a single focused day. Print it, check items off, and by tonight you will have a Claude setup that is genuinely yours.

Morning (Hours 1-4)

- [] Install Claude Code globally with npm
- [] Authenticate with your Claude subscription or API key
- [] Create `~/.claude/CLAUDE.md` with global preferences (or sync from team dotfiles repo)
- [] Verify global layer: personal standards, communication style, universal code conventions
- [] Create project `CLAUDE.md` with stack, architecture, build commands (project-specific only — do not duplicate globals)
- [] Create `.claude/rules/` with team standards: `security-rules.md`, `testing-standards.md`, `api-conventions.md`
- [] Run Claude on your project — ask it to build something. Note what it gets wrong
- [] For each issue, decide: global fix, project fix, or rules fix. Update the right layer. Repeat 2-3 times
- [] Create your first skill: the workflow you do most often (endpoint, component, etc.)
- [] Create two more skills: code-review and test-writer

Afternoon (Hours 5-8)

- [] Add 2-3 slash commands: `/commit`, `/review-pr`, and one project-specific
- [] Set up auto-format hook (PostToolUse on Edit/Write)

-
- [] Set up main-branch protection hook (PreToolUse blocks edits on main)
 - [] Configure one MCP server (database recommended)
 - [] Create reviewer agent (Opus model, critical persona)
 - [] Create debugger or planner agent
 - [] Test the full workflow: write feature -> auto-format -> review -> commit
 - [] Commit .claude/ directory to git (commands, skills, agents, rules, settings.json)
 - [] Verify .gitignore includes settings.local.json and auto-memory/
-

That is it. Not 100 prompts. Not a 14-chapter theory book. Eight hours of focused configuration that will change every coding session from tomorrow onward.

The engineers who get the most from AI tools are not the ones who write the cleverest prompts. They are the ones who invest in the configuration layer — who treat Claude not as a chatbot to be prompted, but as a team member to be onboarded.

You have done that onboarding today. Clone the companion repo to get every template from this book as ready-to-use files:

github.com/apronotti/claude-in-one-day

Now go ship something.

CHAPTER A

COMPLETE CLAUDE.md TEMPLATE

Copy this template. Delete everything that does not apply. Add your specifics. A good CLAUDE.md is 40-80 lines, not 400.

```
# Project: [name]

## Quick Facts
- Stack: [language, framework, version]
- Database: [type, ORM]
- Test: `[test command]`
- Lint: `[lint command]`
- Build: `[build command]`
- Deploy: `[deploy command or N/A]`

## Directory Structure
- app/[folder]/ -> [purpose]
- app/[folder]/ -> [purpose]
- tests/          -> Mirrors app/

## Architecture Decisions
- [Key pattern and WHY]
- [Key constraint and WHY]
- [Integration pattern]

## Code Conventions
- [Naming convention]
- [Error handling pattern]
- [Import ordering]
- [State management approach]

## Do NOT
- [Common antipattern in your codebase]
- [Thing Claude tends to get wrong]
- [Deprecated approach still in code]

## Before Committing
- [ ] Tests pass
- [ ] Lint passes
- [ ] [Project-specific check]
- [ ] [Project-specific check]
```

CHAPTER B

SKILL LIBRARY STARTER KIT

Five ready-to-customize skills. Adapt the patterns to your stack.

Structured Commit Workflow — `commit`

```
---
name: commit
description: >
  Create a well-structured commit following
  conventional commit format.
---

# Commit Workflow
1. Run tests: `pytest`
2. Run lint: `ruff check .`
3. Stage changes: review diff carefully
4. Write commit message:
   - type(scope): description
   - Types: feat, fix, refactor, test, docs
   - Keep subject under 72 chars
   - Add body for non-obvious changes
5. If tests or lint fail, fix before commit
```

Code Review Checklist — `code-review`

```

---
name: code-review
description: >
  Review code against project standards.
  Use when reviewing PRs, before commits,
  or when asked to check code quality.
---

# Code Review
For each changed file, verify:
1. Follows project architecture (CLAUDE.md)
2. Error handling is complete and typed
3. No security issues (injection, XSS, auth)
4. Tests cover happy path + error cases
5. No console.log or debug artifacts
6. Naming matches project conventions

Output format:
- MUST FIX: blocking issues
- SHOULD FIX: improvements
- NITPICK: style suggestions

```

Test Generation — `test-writer`

```

---
name: test-writer
description: >
  Generate tests matching project conventions.
  Use when asked to add tests or test coverage.
---

# Test Writing
- Framework: pytest
- Location: tests/ mirroring app/
- Naming: test_[module].py
- Structure: class TestX with test_ methods
- Always test: happy path, error cases,
  edge cases, validation failures
- Mock external deps, never mock the SUT
- Use factory functions for test data

```

Debug Workflow — `debug`

```
---
name: debug
description: >
  Systematic debugging workflow. Use when
  tracking down bugs, errors, or unexpected
  behavior.
---

# Debug Workflow
1. Reproduce: find the minimal steps
2. Isolate: narrow to file and function
3. Inspect: read the code path carefully
4. Hypothesize: form a theory before fixing
5. Verify: write a failing test first
6. Fix: make the minimal change
7. Confirm: run full test suite

Do NOT:
- Shotgun debug (random changes)
- Fix symptoms without understanding cause
- Skip writing a regression test
```

Code Explainer — explain-code

```
---
name: explain-code
description: >
  Explain code with diagrams and analogies.
  Use when explaining how code works.
---

When explaining code, always include:
1. Start with an everyday analogy
2. Draw ASCII diagram showing the flow
3. Walk through step by step
4. Highlight one common gotcha
```

CHAPTER C

RESOURCES AND LINKS

Book Companion Repository

All templates, skills, commands, agents, hooks, and setup scripts from this book — ready to clone and customize:

github.com/apronotti/claude-in-one-day

Official Documentation

- Claude Code Docs — code.claude.com/docs
- Skills Documentation — code.claude.com/docs/en/skills
- Anthropic Prompt Engineering — docs.claude.com
- MCP Specification — modelcontextprotocol.io

Community Resources

- Claude Code Best Practices (GitHub) — community-maintained patterns and tips
- Awesome MCP Servers — curated list of MCP server implementations
- Claude Code Showcase — example project configurations with hooks, skills, agents

Learning

- Claude Code in Action — Anthropic's free official course
- Anthropic Discord — active community for troubleshooting

This book was written to be read once and referenced often. Come back to the checklists and templates. And remember: the best configuration is the one you actually use.

— END —